

UNIVERSIDADE CATÓLICA DE BRASÍLIA

Curso de Sistemas de Informação | Disciplina de Banco de Dados

SISTEMA DE GESTÃO PARA CLÍNICA ODONTOLÓGICA

Artigo Técnico — Disciplina de Banco de Dados

Antônio Joaquim Alves
Cayo do Prado Santos
Dankley Meireles de Lima
Felipe de Sousa Lopes

Brasília – DF · 2026

RESUMO

Este artigo descreve o desenvolvimento de um sistema de gestão para clínica odontológica utilizando banco de dados relacional MySQL, com integração entre backend em Python (Flask) e frontend em HTML, CSS e JavaScript. O sistema contempla o gerenciamento de consultas, pacientes, dentistas, estoque de insumos e movimentações financeiras. A modelagem do banco de dados seguiu as etapas conceitual, lógica e física, resultando em dez tabelas relacionadas com uso de triggers, views, stored procedures, functions, índices e controle de acesso por usuários. A aplicação demonstra o domínio dos principais recursos de um SGBD em um contexto real e relevante para o Distrito Federal, onde há elevada concentração de profissionais odontológicos.

Palavras-chave: banco de dados relacional; MySQL; clínica odontológica; Flask; gestão de sistemas.

1. INTRODUÇÃO

O Distrito Federal apresenta um dos maiores índices de dentistas por habitante do Brasil. Segundo dados do Conselho Federal de Odontologia (CFO), há aproximadamente 9.343 cirurgiões-dentistas registrados no CRO-DF, resultando em uma proporção de 1 profissional para cada 326 habitantes — quatro vezes acima da recomendação da Organização Mundial da Saúde (OMS), que sugere 1 para cada 1.200 habitantes.

Diante desse cenário, muitos dentistas, especialmente os recém-formados, registram consultas e pacientes em papéis avulsos ou utilizam softwares dispendiosos e incompatíveis entre si. Com o objetivo de democratizar o acesso à tecnologia de gestão clínica, este trabalho propõe o desenvolvimento de um sistema integrado de gerenciamento para clínicas odontológicas, fundamentado em banco de dados relacional MySQL e acessível via interface web.

O sistema permite o cadastro e controle de pacientes, agendamento de consultas, gestão de estoque de insumos, controle financeiro e emissão de relatórios, promovendo eficiência operacional e segurança no armazenamento de dados clínicos.

2. OBJETIVOS

2.1 Objetivo Geral

Desenvolver um sistema de gestão para clínica odontológica com banco de dados relacional MySQL, integrando frontend e backend, demonstrando o domínio dos principais recursos de um SGBD.

2.2 Objetivos Específicos

- Modelar o banco de dados nas etapas conceitual, lógica e física, aplicando normalização e integridade referencial;
- Implementar triggers, views, stored procedures, functions e índices devidamente justificados;
- Definir controle de acesso com usuários distintos, sem uso do root;
- Criar IDs customizados para dados críticos, como consultas, utilizando function específica;
- Desenvolver API REST em Python/Flask que se comunica com o banco de dados;
- Construir interface web que exiba dados das views e permita agendamento via stored procedure.

3. METODOLOGIA

O desenvolvimento do sistema seguiu o ciclo de vida clássico de projetos de banco de dados: levantamento de requisitos, modelagem conceitual, modelagem lógica, modelagem física, implementação, integração e testes.

Na fase de levantamento, foram identificados os processos essenciais de uma clínica odontológica: cadastro de usuários (dentistas e pacientes), agendamento de consultas, controle de estoque e gestão financeira. A partir daí, definiu-se o Modelo Entidade-Relacionamento (MER), que evoluiu para o modelo lógico relacional e, finalmente, para o script DDL em MySQL.

Para o backend, utilizou-se Python com o microframework Flask, expondo endpoints REST que consomem o banco de dados via driver PyMySQL. O frontend foi desenvolvido com HTML5, CSS3 e JavaScript puro, garantindo leveza e compatibilidade. A comunicação entre as camadas segue o padrão cliente-servidor, com a interface consultando as views do banco de dados para exibição dos dados.

4. DESCRIÇÃO DO SISTEMA

4.1 Funcionalidades

- Dashboard com indicadores: consultas ativas, total de pacientes, receita paga e alertas de estoque;

- Agenda completa exibindo dados da view vw_agenda_completa;
- Agendamento de nova consulta via stored procedure sp_agendar_consulta;
- Listagem de pacientes com histórico hospitalar e endereço;
- Visualização do corpo clínico com CRO e especialização;
- Controle financeiro com status de pagamento (receitas e despesas);
- Controle de estoque com alertas de validade (vencido, atenção, OK);
- Relatório de faturamento por dentista via view vw_faturamento_dentistas.

4.2 Tecnologias Utilizadas

A tabela a seguir apresenta as tecnologias utilizadas em cada camada do sistema com suas respectivas justificativas.

Camada	Tecnologia	Justificativa
Banco de Dados	MySQL 8.0	SGBD relacional robusto, amplamente adotado, com suporte completo a triggers, procedures, views e controle de acesso
Backend	Python 3 + Flask	Linguagem de alta legibilidade; Flask é leve e adequado para APIs REST em projetos acadêmicos e protótipos
Conector BD	PyMySQL	Driver Python nativo para MySQL, sem dependências externas complexas
Frontend	HTML5 + CSS3 + JS	Tecnologias padrão da web; sem frameworks adicionais para reduzir complexidade de configuração
CORS	Flask-CORS	Necessário para comunicação entre frontend (porta 80) e backend (porta 5000) em ambiente local

5. MODELAGEM DO BANCO DE DADOS

5.1 Diagrama Entidade-Relacionamento (DER)

O Diagrama Entidade-Relacionamento abaixo representa a estrutura conceitual do banco de dados, evidenciando as entidades, seus atributos e os relacionamentos entre elas. A entidade usuarios é a base do sistema, da qual herdam as especializações cliente e dentista — padrão conhecido como herança por delegação (tabela-pai + tabela-filha com FK compartilhando a PK).

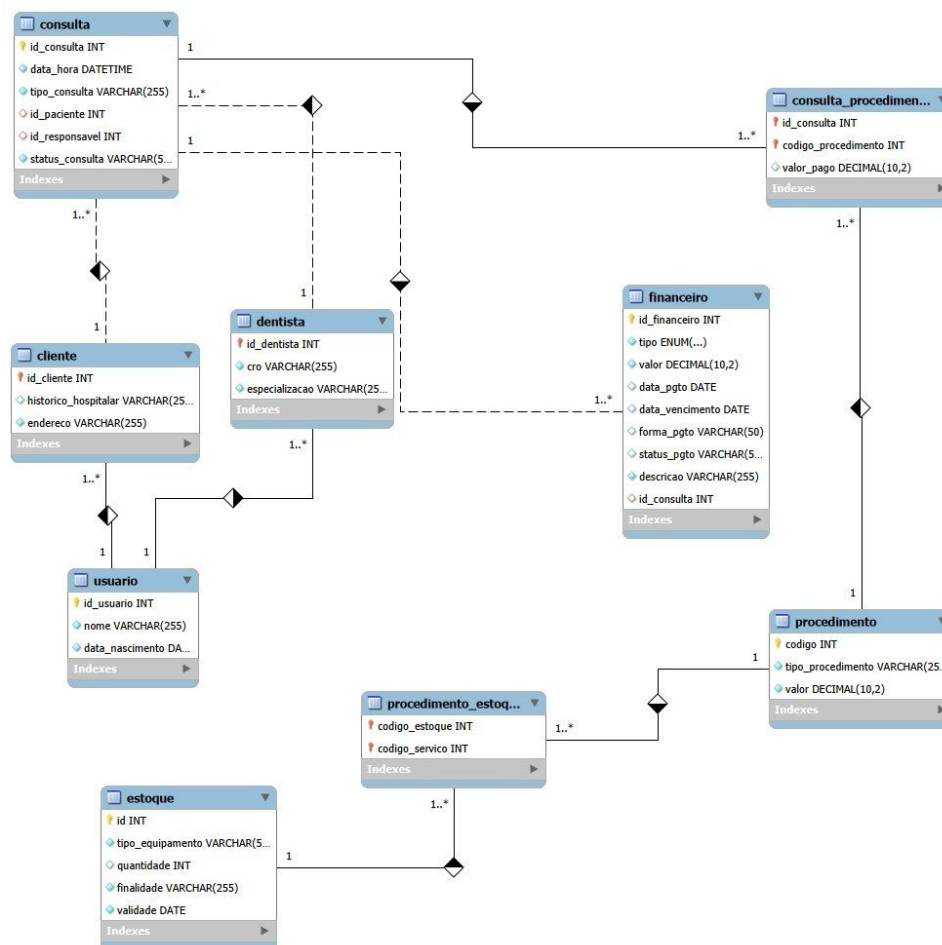


Figura 1 — Diagrama Entidade-Relacionamento (MER) do banco clinica_odontologica

5.2 Entidades e Relacionamentos

O banco de dados clinica_odontologica é composto por dez tabelas que modelam os processos clínicos, administrativos e financeiros da clínica, conforme descrito na tabela abaixo.

Tabela	Descrição
grupos_usuarios	Agrupa usuários por perfil de acesso (Administrador, Dentista, Recepcionista). Tabela obrigatória para controle de permissões.
usuarios	Entidade-base de todos os usuários do sistema (login, senha, nome, grupo). Tabela obrigatória para autenticação e rastreabilidade.
cliente	Especialização de usuarios com histórico hospitalar e endereço do paciente.
dentista	Especialização de usuarios com CRO e especialização clínica do profissional.

consulta	Agendamentos com data, tipo, status, paciente e dentista responsável. IDs gerados por function customizada.
procedimento	Catálogo de procedimentos odontológicos com valor.
consulta_procedimento	Tabela associativa N:N entre consulta e procedimento.
financeiro	Registros de receitas e despesas vinculados ou não a consultas.
estoque	Controle de insumos com quantidade e validade.
procedimento_estoque	Associação N:N entre procedimento e estoque (consumo de insumos por procedimento).

5.3 Índices

Dois índices foram criados com justificativa técnica explícita, visando otimizar operações de alto volume de leitura no sistema.

Índice	Tabela / Coluna	Justificativa
idx_consulta_data	consulta(data_hora)	Consultas de agenda filtram massivamente por data/hora. O índice reduz o custo de varredura de $O(N)$ para $O(\log N)$, essencial em operações frequentes de leitura da agenda.
idx_financeiro_status	financeiro(status_pgto)	Relatórios financeiros filtram constantemente por status de pagamento (Pago/Pendente). O índice acelera JOINS e cláusulas WHERE sobre essa coluna.

5.4 Triggers

Foram implementadas duas triggers com funções distintas, garantindo consistência dos dados de forma automática e transparente para a aplicação.

Trigger	Evento	Função
tr_baixa_estoque_automatica	AFTER INSERT em consulta_procedimento	Quando um procedimento é vinculado a uma consulta, desconta automaticamente os insumos utilizados da tabela estoque, garantindo consistência sem intervenção manual da aplicação.
tr_impedir_alteracao_cancelada	BEFORE UPDATE em consulta	Impede qualquer alteração em consultas com status

		"Cancelada", lançando erro via SIGNAL SQLSTATE. Protege a integridade do histórico clínico e auditabilidade das operações.
--	--	--

5.5 Views

Duas views foram criadas para simplificar consultas complexas, servindo como interface de leitura consolidada para o frontend. Essa abordagem concentra a lógica de negócio no banco de dados e reduz a quantidade de queries emitidas pela camada de aplicação.

View	Descrição	Uso no Sistema
vw_agenda_completa	JOINS entre consulta, usuarios (paciente e dentista) e financeiro, exibindo dados consolidados da agenda.	Exibida diretamente na tela de Agenda do frontend; elimina múltiplas queries na camada de aplicação.
vw_faturamento_dentistas	Agrega total de atendimentos e receita gerada por dentista, filtrando apenas pagamentos confirmados.	Alimenta o relatório de faturamento no Dashboard; concentra lógica de negócio no banco de dados.

A figura abaixo exibe a view vw_agenda_completa sendo consultada diretamente no MySQL Workbench, confirmando a exibição correta de dados consolidados.

```
174 • select * from usuario;
175
```

	id_usuario	nome	data_nascimento
▶	1	Cayo Santos	2000-05-15
	2	Antônio Joaquim	1985-10-20
	3	Ana Oliveira	1992-03-08
	4	Ricardo Silva	1978-12-12
	5	Mariana Costa	1995-07-25
	6	Lucas Pereira	1988-01-30
*	NULL	NULL	NULL

Figura 2 — Resultado da consulta `SELECT * FROM vw_agenda_completa` no MySQL Workbench

5.6 Stored Procedures e Functions

O sistema utiliza procedures e functions para encapsular lógica de negócio crítica diretamente no banco de dados, garantindo consistência independentemente da camada de aplicação que fizer a chamada.

Objeto	Tipo	Descrição e Justificativa
fn_gerar_id_consulta()	Function	Gera ID customizado no formato "CON{ANO}-{SEQUENCIAL}" (ex.: CON2026-00001). Utiliza COUNT por ano para garantir unicidade e rastreabilidade temporal. O formato semântico permite identificar o ano da consulta pelo ID, impossível com AUTO_INCREMENT simples.
sp_agendar_consulta(...)	Procedure	Encapsula a lógica de agendamento: chama fn_gerar_id_consulta() e insere o registro na tabela consulta. Garante que o ID seja sempre gerado de forma padronizada e que a procedure seja o único ponto de entrada para novos agendamentos.
sp_criar_usuario(...)	Procedure	Centraliza a criação de usuários com validação básica, facilitando futuras expansões como hash de senha com bcrypt. Evita inserções diretas na tabela usuarios sem passar pela validação.

O uso de function para geração de ID foi adotado especificamente para dados críticos (consultas), pois permite rastreabilidade temporal e padrão semântico. O AUTO_INCREMENT foi mantido para tabelas de suporte (grupos, procedimentos, estoque, financeiro), onde a rastreabilidade customizada não é necessária e o incremento simples é suficiente — conforme justificado no script DDL.

5.7 Usuários e Controle de Acesso

O acesso ao banco de dados é proibido via root. Foi criado o usuário app_odontoped_user com permissões granulares, separando as responsabilidades de leitura e escrita.

Usuário	Permissões	Justificativa
app_odontoped_user @localhost	SELECT, INSERT, UPDATE, DELETE no schema completo	Utilizado pelo backend Flask para todas as operações CRUD. Não possui permissão de DROP, CREATE ou ALTER, impedindo alterações estruturais acidentais via

		aplicação.
app_odontoped_user @localhost	SELECT exclusivo nas views	Nível de acesso destinado a consultas de leitura pura do frontend, sem possibilidade de modificação de dados. A separação de privilégios segue o princípio do menor privilégio.

Os grupos de usuários da aplicação (tabela grupos_usuarios) definem três perfis de acesso: Administrador (acesso total ao sistema), Dentista (acesso a prontuários e agenda) e Recepcionista (acesso a cadastros e agendamentos). Essa separação permite implementar lógica de autorização na camada de aplicação com base no campo id_grupo da tabela usuarios.

6. EVIDÊNCIAS DE EXECUÇÃO

A figura abaixo apresenta o log de execução completo do script SQL no MySQL Workbench, confirmando que todas as instruções DDL e DML foram executadas sem erros.

#	Time	Action	Message	Duration / Fetch
1	22:00:50	drop database if exists clinica_odontologica	9 rows(s) affected	0.003 sec
2	22:00:50	create database clinica_odontologica	1 row(s) affected	0.016 sec
3	22:00:50	use clinica_odontologica	0 rows(s) affected	0.000 sec
4	22:00:50	create table usuarios (id_usuario int auto_increment primary key, nome varchar(255) not null, data_nascimento date not null)	0 rows(s) affected	0.032 sec
5	22:00:50	create table cliente (id_cliente int primary key, historico_hospitalar varchar(255), endereco varchar(255) not null, foreign key (id_cliente) references usuarios(id_usuario))	0 rows(s) affected	0.031 sec
6	22:00:50	create table procedimento (codigo int auto_increment primary key, tipo_procedimento varchar(255) not null, valor decimal(10,2) not null)	0 rows(s) affected	0.047 sec
7	22:00:50	create table dentista (id_dentista int primary key, cns varchar(255) unique not null, especializacao varchar(255) not null, foreign key (id_dentista) references usuarios(id_usuario))	0 rows(s) affected	0.078 sec
8	22:00:50	create table consulta (id_consulta int auto_increment primary key, data_hora datetime not null, tipo_consulta varchar(255) not null, id_paciente int, id_responsavel int)	0 rows(s) affected	0.078 sec
9	22:00:50	create table financeiro (id_financeiro int auto_increment primary key, tipo_enum(enum('Receita', 'Despesa')) not null, valor decimal(10,2) not null, data_pago date, data_vencimento date)	0 rows(s) affected	0.047 sec
10	22:00:50	create table estoque (id int auto_increment primary key, tipo_enum(enum('Medicamento', 'Material')) not null, quantidade int, validade varchar(255) not null, validade_data date)	0 rows(s) affected	0.031 sec
11	22:00:51	create table procedimento_estoque (codigo_procedimento int, codigo_estoque int, codigo_servico int, primary key (codigo_procedimento, codigo_estoque), foreign key (codigo_procedimento) references procedimento(codigo), foreign key (codigo_estoque) references estoque(id))	0 rows(s) affected	0.047 sec
12	22:00:51	create table consulta_procedimento (id_consulta int, codigo_procedimento int, valor_pago decimal(10,2), primary key (id_consulta, codigo_procedimento), foreign key (id_consulta) references consulta(id_consulta), foreign key (codigo_procedimento) references procedimento(codigo))	0 rows(s) affected	0.031 sec
13	22:00:51	insert into usuarios (nome, data_nascimento) values ('Cauê Santos', '2000-05-10'), ('Fidelino Jacques', '1985-10-20'), ('Ana Oliveira', '1992-03-05'), ('Ricardo Silva', '1990-08-15')	4 rows(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
14	22:00:51	insert into cliente (id_cliente, historico_hospitalar, endereco) values (1, 'CHOC OF 12345 - Odonitoria'), (2, 'CHOC OF 67890 - Implantodontia')	2 rows(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
15	22:00:51	insert into cliente (id_cliente, historico_hospitalar, endereco) values (3, 'Tienhuna alergia detectada', 'Taguatinga Norte, DF'), (4, 'Hipertensão controlada', 'Cajadão')	4 rows(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
16	22:00:51	insert into estoque (tipo_enum, quantidade, validade) values ('Lentes de Látex', 100, 'Proteção Individual', '2027-12-31'), ('Máscara Descartável', 50, 'Proteção Individual', '2027-12-31')	2 rows(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
17	22:00:51	insert into procedimento (tipo_procedimento, valor) values ('Urgência Completa', 150.00), ('Extração Simples', 250.00), ('Restauração de Resina', 180.00), ('Aplicação de Flúor', 50.00)	4 rows(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.016 sec
18	22:00:51	insert into consulta (tipo_consulta, id_usuario, id_dentista, id_responsavel, status_consulta) values ('2025-04-20 09:00:00', 'Receita', 3, 1, 'Agendada'), ('2025-04-20 10:00:00', 'Receita', 4, 2, 'Em andamento')	2 rows(s) affected Records: 2 Duplicates: 0 Warnings: 0	0.000 sec
19	22:00:51	insert into financeiro (tipo, valor_pago, data_vencimento, forma_pagto, descricao, id_consulta) values ('Receita', 150.00, '2025-04-20', '2025-04-20', 'Receita de consulta', 1)	1 rows(s) affected Records: 1 Duplicates: 0 Warnings: 0	0.000 sec
20	22:00:51	insert into procedimento_estoque (codigo_procedimento, codigo_estoque, valor_pago) values (1, 1), (2, 1), (4, 2), (3, 3)	4 rows(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.016 sec
21	22:00:51	insert into consulta_procedimento (id_consulta, codigo_procedimento, valor_pago) values (1, 1), (2, 2), (2, 250.00), (5, 2), (250.00)	4 rows(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec
22	22:00:51	select u.nome as Dentista, count(id_consulta) as Total_Abandonamentos, sum(valor) as Receita_Gerada from Consulta c join Dentista d on c.id_responsavel = d.id_dentista group by d.id_dentista	1 rows(s) returned	0.000 sec / 0.000 sec
23	22:00:51	select u.nome as Paciente, c.endereco, c.historico_hospitalar from Cliente c join Usuario u on c.id_usuario = u.id_usuario where c.endereco like '%Taguatinga%'	2 rows(s) returned	0.000 sec / 0.000 sec
24	22:00:51	select tipo_procedimento, quantidade, validade from estoque where validade < '2025-01-01' order by validade asc LIMIT 5, 1000	2 rows(s) returned	0.000 sec / 0.000 sec
25	22:00:51	update consulta set status_consulta = 'Finalizada' where id_consulta = 1	1 rows(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
26	22:00:51	update cliente set historico_hospitalar = concat(historico_hospitalar, ' Nova consulta. Paciente estável') where id_cliente = 3	1 rows(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec

Figura 3 — Log de execução do script_clinica.sql no MySQL Workbench (26 instruções executadas com sucesso)

O log evidencia a execução das operações de criação de banco de dados, tabelas, inserção de dados, criação de procedimentos/triggers, execução das queries de exemplo e atualização de registros — cobrindo todo o ciclo de vida das operações especificadas.

7. CONCLUSÃO

O sistema de gestão para clínica odontológica foi desenvolvido com sucesso, contemplando todos os requisitos técnicos estabelecidos: estrutura relacional

completa com dez tabelas (incluindo as obrigatórias usuarios e grupos_usuarios), triggers de automatização e proteção de dados, views para abstração de consultas complexas, stored procedures e functions para encapsulamento de lógica de negócio — incluindo geração customizada de IDs para dados críticos —, índices justificados para performance, e controle de acesso sem uso do root.

A integração entre o banco de dados MySQL, a API REST em Python/Flask e o frontend em HTML/CSS/JS demonstrou a viabilidade de construir soluções completas e funcionais com tecnologias abertas e acessíveis. O uso de mock data no frontend garante que a aplicação possa ser demonstrada mesmo sem conexão ativa ao banco, facilitando apresentações em ambiente sem servidor.

Como trabalhos futuros, sugere-se a implementação de autenticação JWT na API, hash de senhas com bcrypt, paginação dos resultados nos endpoints, e possível migração do frontend para um framework como React ou Vue.js para maior escalabilidade.

8. REFERÊNCIAS

ELMASRI, Ramez; NAVATHE, Shamkant B. *Sistemas de banco de dados*. 7. ed. São Paulo: Pearson, 2019.

DATE, C. J. *Introdução a sistemas de bancos de dados*. 8. ed. Rio de Janeiro: Campus, 2004.

MySQL. *MySQL 8.0 Reference Manual*. Disponível em: <https://dev.mysql.com/doc/refman/8.0/en/>. Acesso em: mai. 2026.

FLASK. *Flask Documentation*. Disponível em: <https://flask.palletsprojects.com/>. Acesso em: mai. 2026.

CONSELHO FEDERAL DE ODONTOLOGIA. *Dados estatísticos*. Disponível em: <https://cfo.org.br/>. Acesso em: mai. 2026.